Lesson 14 · AI Code Review & Issue Detection

PDF: this lesson · **Install (Linux · macOS · Windows):** guide · PDF

> **Part of local-ai-lab** - a hands-on course for building local AI.

> **Course home:** https://nikolareljin.github.io/local-ai-lab/ **Source:** https://github.com/nikolareljin/local-ai-lab

> **Lessons:** 1 · RAG → 2 · MCP → 3 · Hybrid retrieval → 4 · RAG safety → 5 · RAG evaluation → 6 · Repo assistant → 7 · LangChain → 8 · LangGraph → 9 · Ollama tools → 10 · Semantic Kernel → 11 · Bedrock → 12 · Google ADK → 13 · AI-assisted testing → **14 · AI code review (you are here)** → 15 · Docs from changes

> **Status: planned.** Outline below; full step-by-step coming later. Star the repo to follow along.

The idea

Code review is where AI is most oversold, and the reason is a measurement problem. Generating forty comments on a pull request is trivial. **Reading forty comments is expensive**, and a reviewer that produces thirty-seven maybes and three real bugs gets muted within a week - after which the three real bugs go unread too.

So this lesson optimises for the opposite of what the demos optimise for: **precision over recall.** A reviewer that reports three real bugs beats one that reports forty candidates, and the way you find out which one you built is to seed known bugs and count what came back.

What you'll learn

- **Scope to the diff plus its call sites** - a changed function is not reviewable without the code that calls it, and the whole repo is not reviewable at all
- **Severity that means something** - if everything is a warning, nothing is
- **A verify pass** - a second model adversarially checking the first one's finding, which is the same shape as the grading node in Lesson 8, applied to claims instead of evidence
- **Suppress what the linter already catches** - duplicating a tool that never sleeps is pure noise
- **Measure it** - seed bugs deliberately, then count found versus invented. Without that number you are guessing

Builds on

Concept	From
a pull request is **untrusted input**	Lesson 4

repo-aware retrieval with citations	Lesson 6
a grade step that can reject its own input	Lesson 8
precision, and how to measure it	**Lesson 14 (this one)**

> **Worth saying out loud:** a diff is text written by someone else, and you are about to feed it to a model with tools. Everything Lesson 4 said about poisoned documents applies here, with the added detail that the author of a pull request may want it to.

Prerequisites

Lesson 1. Lesson 4 is close to required in spirit, and Lesson 6 supplies the repo-aware half. Language-agnostic - the subject is the review, not the syntax.

Next lesson

Lesson 15 · Documentation from Sprint Changes →
